

# Optimizer Amalgamation



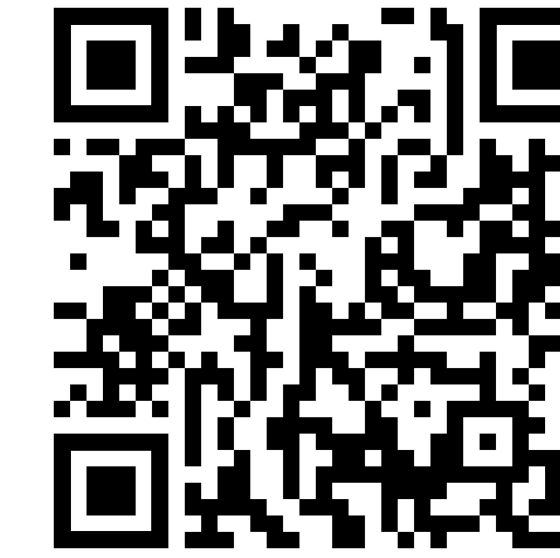
Tianshu Huang<sup>1,2</sup>, Tianlong Chen<sup>1</sup>, Sijia Liu<sup>3</sup>, Shiyu Chang<sup>4</sup>, Lisa Amini<sup>5</sup>, Zhangyang Wang<sup>1</sup>

<sup>1</sup>University of Texas at Austin, <sup>2</sup>Carnegie Mellon University, <sup>3</sup>Michigan State University, <sup>4</sup>University of California, Santa Barbara, <sup>5</sup>MIT-IBM Watson AI Lab, IBM Research

tianshu@cmu.edu, {tianlong.chen, atlaswang}@utexas.edu, liusiji5@msu.edu, chang87@ucsb.edu, lisa.amini@ibm.com



VITA Group



Github Page



UC SANTA BARBARA

## Motivation

Meta-training optimizers (“Learning to Optimize” [1]) from scratch is quite difficult. On the other hand, much attention has been directed towards developing a diverse body of analytical optimizers. Can we use these optimizers to meta-train a stronger optimizer?

## Contributions

**Optimizer Amalgamation** We define “Optimizer Amalgamation:” how do we best combine multiple optimizers into a single stronger optimizer?

**Mean:** add loss (“distillation loss” - l2 distance) for each optimizer

$$\mathcal{L}_{\text{mean}}(\mathbf{x}; \theta_i; \phi) = \mathcal{L}_{\text{meta}}(\mathbf{x}; \theta_i^{(P)}; \phi) + \alpha \frac{1}{N} \sum_{i=1}^N \frac{1}{|T|} \sum_{i=1}^{|T|} \log \left\| \theta_i^{(P)} - \theta_i^{(T_i)} \right\|_2$$

**Min-max:** use loss of the furthest optimizer

$$\mathcal{L}_{\text{min-max}}(\mathbf{x}; \theta_i; \phi) = \mathcal{L}_{\text{meta}}(\mathbf{x}; \theta_i^{(P)}; \phi) + \alpha \frac{1}{N} \sum_{i=1}^N \max_{T \in T} \log \left\| \theta_i^{(P)} - \theta_i^{(T)} \right\|_2$$

**Choice:** train an intermediate optimizer to choose the best optimizer at each step

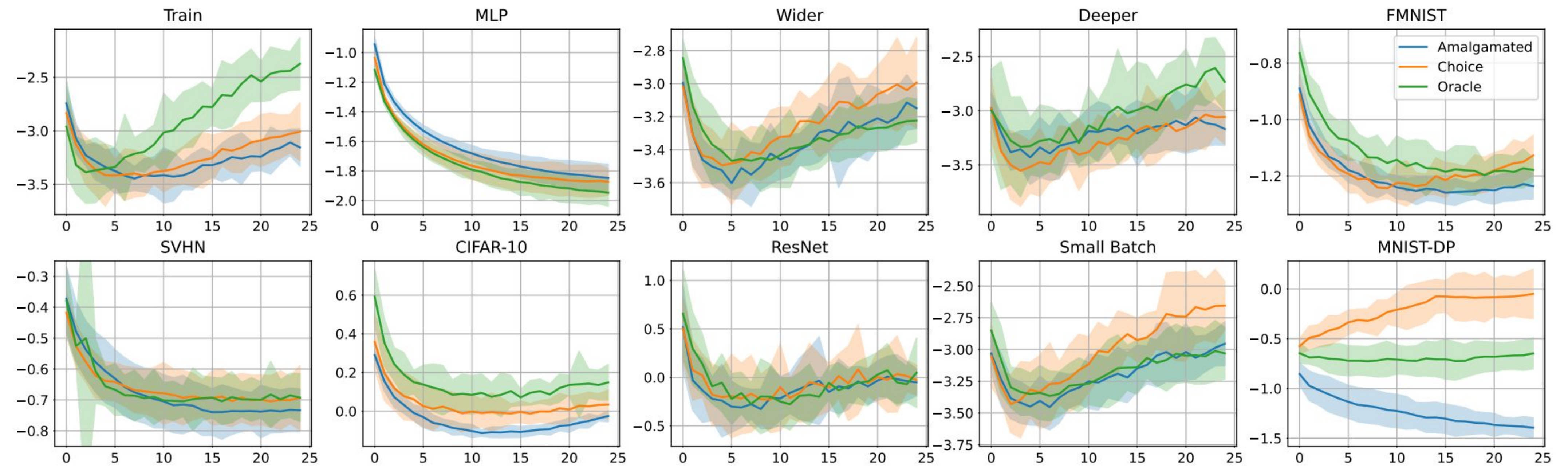
$$\mathcal{L}_{\text{choice}} = \mathcal{L}_{\text{meta}}(\phi; \mathbf{x}) + \alpha \frac{1}{N} \sum_{i=1}^n \log \left\| \theta_i^{(P)} - \theta_i^{(C)} \right\|_2$$

**Perturbations** We use adversarial and random perturbations to alleviate high variance during training.

**Experiments** We report extensive experiments evaluating 8 replicates for each method, 10 problems per replicate, and 10 runs per problem.

## Related Work

- [1] Chen et al, “Learning to Optimize: A Primer and A Benchmark”.
- [2] Chen et al, “Training Stronger Baselines for Learning to Optimize”, NeurIPS 2020



## Results

**Optimizer Pool** We used 6 optimizers in our pool: Adam, Momentum, RMSProp, SGD, AddSign, and PowerSign.

**Above** Our amalgamated optimizer is consistently as good as the “oracle” optimizer, which is the analytical optimizer with the best performance on each problem, and the “choice” optimizer, which is the optimizer trained as an intermediate step in optimal choice amalgamation.

**Below** With the same amount of training, our method consistently outperforms existing learning to optimize methods.

**Right** Appropriately scaled perturbations can significantly reduce variance during training.

